

Python asyncio 异步编程

asyncio 是 Python3.4+ 版本中内置的异步 I/O 库，提供了一套完整的异步编程框架，用于编写高效的并发程序。核心思路是在单个线程中通过事件循环(Event Loop)来实现并发任务调度，避免了多线程、多进程并发的问题，特别适合 I/O 密集型场景，比如：文件读写、数据库操作、LLM 生成等领域；

并行：真正的同时执行，一般需要多核处理器

同一时刻，多个任务分别在不同核上独立运行，彼此互不干涉；

并发：交替执行多任务，看似是同时执行

同一时刻，只有一个任务在真正的执行，其它任务均处于挂起(等待)状态；

asynio：协程 — 单线程异步 I/O （用户态并发）

单线程：基于协程实现，运行在单个线程内，属于用户态的非抢占式并发；

非抢占式：协程的执行权切换完全由程序自身控制，通过 `await` 关键字来交出执行权，操作系统感受不到协程的概念存在，不会强制调度；

单线程异步：所有协程在当个线程中轮转执行，当某个协程遇到 I/O 阻塞的时候，会通过 `await` 关键字主动挂起，将执行权交给其它就绪的协程。

多线程(threading 模块)：线程级并发 — 内核态抢占式调度

多个线程运行在同一进程的地址空间中，共享进程的资源，比如：内存、文件句柄等等

抢占式：线程的执行权切换是由操作系统的内核调度控制的，操作系统会根据时间片、线程优先级等信息强制将 CPU 执行权从一个线程切换到另外一个线程。

全局解释器锁(GIL)：再 CPython 解释器中，GIL 锁会保证同一时刻只会有一个线程能够执行 python 字节码，这表示在 CPU 密集型任务中无法真正的实现并行，只有 I/O 密集型任务重，线程由于 I/O 阻塞释放 GIL 锁，才能发挥多线程的优势。

多进程(multiprocessing 模块)：进程并发 — 独立内存空间 + 内核调度

CPython 中真正意义上的并行执行(多核 CPU 下可以运行多个进程)

独立资源：每个进程都有独立的内存空间、Python 解释器等，默认情况下进程之间不共享资源；

无 GIL 限制：每个进程独立 GIL 锁，多进程之间互不干涉，可实现 CPU 密集型任务的并行加速；

内核调度：进程的创建、销毁以及切换等操作均需要由操作系统内核执行，属于抢占式调度。

核心关键字

关键字	作用说明
async def	用来定义异步函数，调用该函数并不会立即执行，而是返回一个协程对象
await	用于暂停当前协程的执行，等待指定异步操作(协程对象等)操作完成，期间会调度其它协程的执行。await 关键字只可以使用在 async def 定义的函数内部
asyncio.create_task	将传入的协程对象封装为可调度的 Task 对象，并理解提交给事件循环调度执行，实现异步执行的并发执行。
asyncio.gather	批量启动多个协程/Task/Future 对象，并等待所有对象的执行完成，最终返回结果列表，可通过参数 return_exceptions=True 控制异常处理逻辑；
asyncio.wait	等待一组协程/Task/Future 对象执行完成，支持超时控制以及返回条件；最终返回已完成和未完成的任务集合。
asyncio.sleep	异步睡眠方法。一般用来替代同步睡眠方法 time.sleep
asyncio.run	程序入口，用来自动管理事件循环逻辑；

多线程案例：

▼ threading demo Python

```

# -*- coding: utf-8 -*-
"""
threading demo
"""
import datetime
import threading
import time
import os

def thread_task(task_name, delay):
    """线程任务：模拟耗时操作 ( I/O 密集型 ) """
    _msg = f"进程id:{os.getpid()} 父进程id:{os.getppid()} 线程id:{threading.current_thread().ident}"
    print(f"[{datetime.datetime.now().strftime('%H:%M:%S.%f')}] {_msg} 线程任务 {task_name} 开始执行，延迟 {delay} 秒")

    # 模拟耗时 ( 此处用time.sleep模拟I/O阻塞，线程会释放GIL，允许其他线程执行 )
    time.sleep(delay)
    print(f"[{datetime.datetime.now().strftime('%H:%M:%S.%f')}] {_msg} 线程任务 {task_name} 执行完成")

```

```

# threading库 多线程模块中默认不支持返回结果，需要通过其它方式来获取返回结果
return f"{task_name} {_msg} 结果"

def main_thread():
    """多线程主函数"""
    print("==== 多线程执行开始 =====")
    start_time = time.time()

    # 1. 创建线程对象
    thread1 = threading.Thread(target=thread_task, args=("任务A", 2), name="Thread-1")
    thread2 = threading.Thread(target=thread_task, args=("任务B", 1), name="Thread-2")
    thread3 = threading.Thread(target=thread_task, args=("任务C", 3), name="Thread-3")

    # 2. 启动所有线程 (启动后线程进入就绪状态，由系统调度执行)
    thread1.start()
    thread2.start()
    thread3.start()

    # 3. 等待所有线程执行完成 (主线程阻塞，直到子线程全部结束)
    thread1.join()
    thread2.join()
    thread3.join()

    end_time = time.time()
    print(f"==== 多线程执行结束 =====")
    print(f"总耗时: {end_time - start_time:.2f} 秒")

if __name__ == "__main__":

```

多进程案例：

▼ multiprocessing demo Python

```

# -*- coding: utf-8 -*-
import datetime
import multiprocessing
import os
import threading
import time

def process_task(task_name, delay, result_queue):
    """进程任务：模拟耗时操作 (CPU 密集型/ I/O 密集型)"""
    _msg = f"进程id:{os.getpid()} 父进程id:{os.getppid()} 线程id:{threading.current_thread().ident}"
    print(f"[{datetime.datetime.now().strftime('%H:%M:%S.%f')}] {_msg} 进程任务 {task_name} 开始执行，延迟 {delay} 秒")

    # 模拟耗时 (CPU 密集型用循环，I/O 密集型用time.sleep，此处统一用sleep)

```

```

    time.sleep(delay)
    print(f"[{datetime.datetime.now().strftime('%H:%M:%S.%f')}] {_msg} 进程任务 {task_name} 执行完成")

# 将结果放入队列 (进程间数据共享方式之一)
result = f"{task_name} {_msg} 结果"
result_queue.put(result)

def main_process():
    """多进程主函数"""
    print("==== 多进程执行开始 =====")
    start_time = time.time()

    # 1. 创建结果队列 (用于进程间传递结果, IPC 机制)
    result_queue = multiprocessing.Queue()

    # 2. 创建进程对象
    process1 = multiprocessing.Process(
        target=process_task,
        args=("任务A", 2, result_queue),
        name="Process-1"
    )
    process2 = multiprocessing.Process(
        target=process_task,
        args=("任务B", 1, result_queue),
        name="Process-2"
    )
    process3 = multiprocessing.Process(
        target=process_task,
        args=("任务C", 3, result_queue),
        name="Process-3"
    )

    # 3. 启动所有进程
    process1.start()
    process2.start()
    process3.start()

    # 4. 等待所有进程执行完成
    process1.join()
    process2.join()
    process3.join()

    # 5. 从队列中获取所有结果
    results = []
    while not result_queue.empty():
        results.append(result_queue.get())
    print(f"所有任务结果: {results}")

    end_time = time.time()
    print(f"==== 多进程执行结束 =====")
    print(f"总耗时: {end_time - start_time:.2f} 秒")

```

```
if __name__ == "__main__":
```

协程案例：

▼ asyncio demo1

Python

```
# -*- coding: utf-8 -*-
import asyncio
import os
import threading
import datetime
import time

async def coro_task(task_name, delay):
    """协程任务：模拟耗时 I/O 操作"""
    _msg = f"进程id:{os.getpid()} 父进程id:{os.getppid()} 线程id:{threading.current_thread().ident}"
    print(f"[{datetime.datetime.now().strftime('%H:%M:%S.%f')}] {_msg} 协程任务 {task_name} 开始执行，延迟 {delay} 秒")

    # 异步睡眠（非阻塞，期间事件循环调度其他协程）
    await asyncio.sleep(delay)
    print(f"[{datetime.datetime.now().strftime('%H:%M:%S.%f')}] {_msg} 协程任务 {task_name} 执行完成")

    # 可直接返回结果
    return f"{task_name} {_msg} 结果"

async def main_coro():
    """协程主函数"""
    print("==== 协程执行开始 =====")
    start_time = time.time()

    # 1. 并发执行多个协程任务（两种方式任选）
    # 方式1：create_task 手动创建任务
    task1 = asyncio.create_task(coro_task("任务A", 2), name="Coro-1")
    task2 = asyncio.create_task(coro_task("任务B", 1), name="Coro-2")
    task3 = asyncio.create_task(coro_task("任务C", 3), name="Coro-3")

    # 等待所有任务完成，获取结果
    result1 = await task1
    result2 = await task2
    result3 = await task3

    # 方式2：gather 批量执行（更简洁，推荐批量任务）
    # results = await asyncio.gather(
    #     coro_task("任务A", 2),
```

```

#     coro_task("任务B", 1),
#     coro_task("任务C", 3)
# )

end_time = time.time()
print(f"==== 协程执行结束 =====")
print(f"任务结果: {result1}, {result2}, {result3}")
print(f"总耗时: {end_time - start_time:.2f} 秒")

if __name__ == "__main__":
    # 启动协程程序 (自动管理事件循环)
    asyncio.run(main_coro())

```

Python

```

# -*- coding: utf-8 -*-

import asyncio
import random
from typing import AsyncGenerator
import threading

async def async_data_generator(cnt: int) -> AsyncGenerator[int, None]:
    thread = threading.current_thread()
    print(f"async_data_generator ----> {thread.name}_{thread.id}")
    for i in range(cnt):
        # 等待 --> 模拟异步操作
        await asyncio.sleep(random.uniform(0.5, 1.5))
        # 产生随机数
        rnd_data = i * 10 + random.randint(1, 9)
        print(f"数据生成: {rnd_data}")
        # 数据返回
        yield rnd_data

async def main():
    thread = threading.current_thread()
    print(f"main ----> {thread.name}_{thread.id}")
    data_iter = async_data_generator(5)
    print(f"迭代器对象: {data_iter}")

    print("开始获取迭代器内的数据.....")
    async for data in data_iter:
        print(f"数据: {thread.name}_{thread.id} ----> {data}")

if __name__ == '__main__':
    asyncio.run(main())

```

